

Integral Algorithm for Generating Anti-Aliasing Circle Based on Bresenham Algorithm

Bowen Fu^{1, a}, Lianqiang Niu^{2, b}

¹ School of Software, Shenyang University of Technology

²School of Information Science and Engineering, Shenyang University of Technology

^aawen28@126.com, ^bniulq@sut.edu.cn

Keywords: anti-aliasing; straight line drawing; circle generating; curve generating; integer operation; pixel-level drawing; Bresenham algorithm; lightness conversion

Abstract. For generating anti-aliasing circle with a high speed and a simple manner, an algorithm which uses only integral operation to convert gray levels of pixels based on modified Bresenham Algorithm is presented. According to the decision parameter established in Bresenham algorithm, the algorithm generated the accurate distances between candidate points and true curve, abandoned the higher-order epsilon with litter effect, and used look-up table or range binary search to calculate the gray levels of the nearest pixels. The new algorithm only modified the local structure of the original algorithm and can generate smooth circle rapidly because of simple structure and only integer operation employed in it. Besides, the new algorithm makes the basic generating algorithm and anti-aliased drawing algorithm can be easily generated under the same framework.

Introduction

Straight line, circle and ellipse are the basic element of computer graphics system, and the improvement for generating speed and display quality of them is very useful. With increasing requirements about display qualities of graphics and images, the better algorithms of anti-aliasing graphics and images are needed widely too. Basic generating methods is Bresenham Algorithm and developed pixel-based algorithms[2-6], but most algorithms which argued generating anti-aliasing graphics focus on straight lines rather than curves. For the problem of anti-aliasing straight line, paper [7] proposed a kind of simple method with high efficiency, which bases on accumulating the slope of line in floating-point number manner directly, and converting them into gray levels of pixels by fixed point decimal operation. Several algorithms for generating anti-aliasing circle are proposed in [8-12]. Two kind of double-step algorithms for generating anti-aliasing circle with integral operation were argued in paper [13], [14] and [15].

This paper analyzed the distances between candidate pixels and true circle, with basing on Bresenham Algorithm, and built an algorithm for generating anti-aliasing circle by only using integer operation. This algorithm is developed from Bresenham Algorithm directly. It adopts the structure of Bresenham Algorithm but does further classification in the local. Since more accurate gray-levels can be gained by using distances from candidate pixels to curve and integral operations in the algorithm, the efficiency and display quality are both satisfactory.

View of Bresenham Algorithm to Generate Circle

For the symmetry of the circle, we only argue the first 1/8 region of a circle which radius is r and its centre is on the origin point. Other regions can be drawn by the symmetry.

As the same as famous algorithm for generating straight line, for a loop to generate the first 1/8 arc, we always move in X-direction, i.e., $x_{i+1}=x_i+1$. And y, there are two possible value for the selection: $y_{i+1}=y_i$ or $y_{i+1}=y_i-1$, shown in Fig. 1.

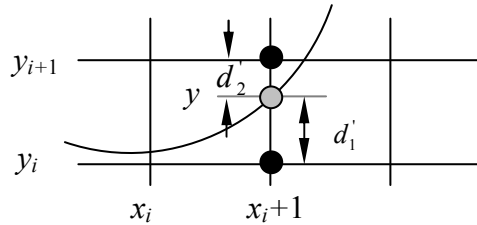


Fig. 1 Candidate pixels at the $(i+1)$ th step

The d'_1 and d'_2 in Fig. 1 have the most same meaning as d_1 and d_2 respectively, but the values of them are different. Define the decision parameter D_i as follow form:

$$d_1 = y_i^2 - y^2 = y_i^2 - r^2 + (x_i + 1)^2, \quad d_2 = y^2 - y_{i+1}^2 = r^2 - (x_i + 1)^2 + (y_i - 1)^2,$$

$$D_i = d_1 - d_2 = 2(x_i + 1)^2 + y_i^2 - (y_i - 1)^2 - 2r^2.$$

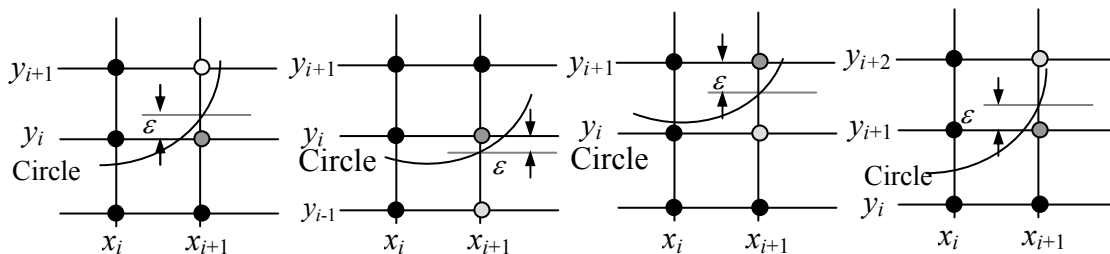
If $D_i < 0$, then $y_{i+1}=y_i$, else $y_{i+1}=y_i-1$. So we get the recursive relation as follow ^{[1][13]}:

$$\begin{cases} D_{i+1} = D_i + 4x_i + 6 & , D_i < 0 \\ D_{i+1} = D_i + 4(x_i - y_i) + 6 & , D_i \geq 0. \\ D_0 = 3 - 2r \end{cases} \quad (1)$$

In order to meet the equation of circle, parameters d_1 , d_2 and D_i are all defined with some forms including square operation, which will be easy to construct iteration relationship. The actual values are not necessary for generating raster circle, but these values may be used to get information of gray level for drawing anti-aliasing circle. While consider that D_i is ascertained by d_1 and d_2 , we can also find out how the values of D_i influence the values of d'_1 and d'_2 reversely.

Analyses and calculations of information for anti-aliased generating circle

While analyzing the relative positions between arc and candidate points (x_i+1, y_i) and (x_i+1, y_i-1) , there are four cases shown in Fig. 2 a)~2 d).



a) $D_i \leq 0, D_i + 2y_i - 1 < 0$ b) $D_i \leq 0, D_i + 2y_i - 1 \geq 0$ c) $D_i > 0, D_i - 2y_i + 1 < 0$ d) $D_i > 0, D_i - 2y_i + 1 \geq 0$

Fig. 2 The distance between arc and target pixel

Theorem 1. Let ε denote accurate distance from theoretical arc to candidate point (x_i+1, y_i) or point (x_i+1, y_i-1) . Then we gain

(1) If $D_i < 0$, then

a) For case $D_i + 2y_i - 1 < 0$, $\varepsilon \approx \frac{-D_i - 2y_i + 1}{4y_i}$; b) For case $D_i + 2y_i - 1 \geq 0$, $\varepsilon \approx \frac{D_i + 2y_i - 1}{4y_i}$.

(2) If $D_i \geq 0$, then

c) For case $D_i - 2y_i + 1 < 0$, $\varepsilon \approx \frac{-D_i - 2(y_i - 1) + 1}{4(y_i - 1)}$; d) For case $D_i - 2y_i + 1 \geq 0$, $\varepsilon \approx \frac{D_i - 2(y_i - 1) + 1}{4(y_i - 1)}$.

Proof. For the case *a*), y of raster circle will not change as a movement has occurred, and pixel $(x_i + 1, y_i)$ should be selected in next step. Since arc is upon the selected point, we obtain $y = y_i - \varepsilon$. So, we get $D_i = d_1 - d_2 = y_i^2 - 2(y_i - \varepsilon)^2 + (y_i - 1)^2 = 4y_i \varepsilon - 2y_i + 1 - 2\varepsilon^2$. Rearrange the formula and yields:

$$\varepsilon = \frac{D_i + 2y_i - 1 + 2\varepsilon^2}{4y_i}. \quad (2)$$

Formula (2) shows the complex of anti-aliasing generating circle and general curve: It will include complicated square root operations if we want to calculate exact value of the ε . So some researchers adopted other ways to avoid this problem [9-14].

In fact, anti-aliasing drawing is a kind of approximate treatment. We notice that $\varepsilon^2 / 2y_i$ is a high-order infinitesimal comparing with ε , so it could be ignored when gray values are converted from distances. Here we gain

$$\varepsilon \approx \frac{D_i + 2y_i - 1}{4y_i}. \quad (3)$$

For this case, the pixels $(x_i + 1, y_i)$ and $(x_i + 1, y_{i+1})$ should be plotted for generating anti-aliasing circle, and the gray level of them are $\varepsilon \times G$ and $G - \varepsilon \times G$. In later algorithm a function $\text{getGray}(\varepsilon)$ is employed to calculate the approximate value of $\varepsilon \times G$.

Similar relationships may established for other cases. Due to space limitations, discussion about (c) and (d) are omitted.

If $d_1 < d_2$, then two cases (a) and (b) can be distinguished by the condition $D_i + 2y_i - 1 \geq 0$. If $d_1 \geq d_2$, the two cases (c) and (d) can be distinguished by the condition $D_i - 2y_i + 1 \geq 0$.

Integral algorithm for anti-aliased generating circle

The algorithm to generate directly the first 1/8 circle arc starting at the point (0, r) is described here. The other regions are obtained by calculating and plotting their symmetric pixels at each step.

Algorithm 1. Integral algorithm for anti-aliased generating circle

```

int  $x \leftarrow 0$ ,  $y \leftarrow r$ ,  $Di \leftarrow 3 - 2r$ ;
while( $x < y$ ) {
  if( $Di < 0$ ) {
    if( $Di + 2y - 1 \geq 0$ ) {
       $g \leftarrow \text{getGray}(Di + 2y - 1, 4y)$ ;  $\text{symPixels}(cx, cy, x, y, 0, -1, g)$ ; }
    else {
       $g \leftarrow \text{getGray}(-Di - 2y + 1, 4y)$ ;  $\text{symPixels}(cx, cy, x, y, 0, 1, g)$ ; }
     $Di \leftarrow Di + 4x + 6$ ; }
  else {
    if( $Di - 2y + 1 \geq 0$ ) {
       $g \leftarrow \text{getGray}(Di - 2y + 1, 4(y - 1))$ ;  $\text{symPixels}(cx, cy, x, y - 1, -2, g)$ ; }
    else {
       $g \leftarrow \text{getGray}(-Di + 2y - 1, 4(y - 1))$ ;  $\text{symPixels}(cx, cy, x, y - 1, 0, g)$ ; }
     $Di \leftarrow Di + 4(x - y) + 10$ ;  $y \leftarrow y - 1$ ; }
}

```

```

    x ← x + 1;
}

```

The point (cx, cy) is the centre of the circle in the algorithm. Function $\text{symPixels}(cx, cy, x, y, \text{off1}, \text{off2}, g)$ is employed to draw pixel (x, y) and its 7 symmetric pixels on the circle by using the centre point (cx, cy) , the offset off1 and off2 , and the gray level g .

Function of the method $\text{getGray}(c, s)$ in Algorithm 1 is to calculate out the pixel lightness (gray level) by weight of c/s , and function of the method $\text{Pixel}(x, y, g)$ is to draw the pixel at the position (x, y) with the gray level g . Usually, we may build up a predefined lookup-table for all integer c and constant integer s , thus any gray level can be gained by searching the table for specified device. This progress costs very little time – commonly only about 5 times operations will be enough.

Analyses and Discussions

Our algorithm is only established on the base of Bresenham Algorithm. At the purpose to explain the main idea of the algorithm, it didn't employee additional skills to improve the efficiency. We use another if-else structure here to divide the cases of if-else structure of Bresenham Algorithm, so one additional comparison operation is taken in each loop comparing with Bresenham algorithm.



a) $r=12$, Our algorithm b) $r=12$, Floating-point algorithm c) $r=18$ (The first 1/4 circle)

Fig. 3. Comparison of qualities between our algorithm and floating-point algorithm

The main time in each loop is consumed at conversion from distance to gray level. It includes 2~5 times basic operation, and $4m \sim 8m$ times basic operations consumed by the binary search, where m is number of gray level. If eight gray levels are employed, it will cost about 12 times operations. This shows that this algorithm is simpler and faster than common floating-point algorithm. Besides, Since only integral operations are invoked in the algorithm, it is suitable for hardware implement. We have tested our algorithm with different parameters, and have drawn the same curve by an algorithm which employs floating-point operations and 256 gray levels and regards accurate distances from selected pixels to true curve as weights of gray levels. The result is excellent similitude, shown in Fig. 3. The distance calculation in our algorithm is very accurate (Only a high-order infinitesimal is ignored), so accurate the pixel lightness information will be also gathered. Comparing two algorithms proposed by paper [8] and [15], which both draw anti-aliasing circle with double-step technique, the number of basic operations consumed by our algorithm is little more than that consumed by them. However, only one or five gray levels can be distinguished in these algorithms. Experiments show that anti-aliasing curve will display in lower quality if gray level is less than 8. Furthermore, the pre-estimation value of gray level adopted by paper [15] can not be changed with curvature of the curve, so it is hard to improve the anti-aliasing quality for practicality.

It should be mentioned that because of ignored the high-order infinitesimal in the algorithm, the shape of anti-aliasing circle will have slight deviation of the senses when the radius of circle is smaller ($r < 15$) and the curvature is big. For this case, we can make y act a control variable for the loop in the algorithm, and divide the loop into two parts. For the small y , a constant compensation should be given to the gray level.

Conclusions

We propose an anti-aliasing algorithm for generating anti-aliasing circle based on Bresenham Algorithm here. The algorithm uses more accurate errors calculated by means of integral basic operations to gain gray levels, and has a very similar structure with Bresenham Algorithm. The derivation of new algorithm is simple, and no any other additional complex concepts or operations are invoked, so it is easy to be implemented both with hardware and high-level language. And also the number of gray levels can be changed by the requirement of image quality. Furthermore, the new algorithm describes a method to make transition to anti-aliasing algorithm from basic generating algorithm of curves.

References

- [1] J. E Bresenham: IEEE Computer Graphics and Application, Vol. 7 (1987) No. 5, p.31.
- [2] V. Boyer, J. J. Bourdin: IEEE Computer Graphics and Application, Vol. 20(2000) No.5, p.67.
- [3] X. H. Lin, T. W. Zhang: Journal of Computer-Aided Design & Computer Graphics, Vol. 18 (2006) No.8, p.1136.
- [4] B. M. Huang, L. Zhang: Journal of Tsinghua University(Sci & Tech), Vol. 46 (2006) No.10, p.1719.
- [5] Y. L. Jia, H. C. Zhang and Y. Z. Jing: Journal of Image and Graphics, Vol. 13 (2008) No. 1, p.158.
- [6] L. Q. Niu, Z. Shao: Journal of Computer-Aided Design & Computer Graphics, Vol. 22(2010) No. 8, p.1286.
- [7] X. Wu: Computer Graphics: Proceedings of SIGGRAPH'91, Vol. 25 (1991) No.4, p.143.
- [8] D. Field: Computer Vision: Graphics. and Image Processing, Vol. 33 (1986) No.1, p.1.
- [9] F. C. Crow: Communications of the ACM, Vol. 20 (1977) No.8, p.799.
- [10] M. L. V. Pitteway, D. J. Watkinson: Communications of the ACM, Vol. 23 (1980) No.11, p.625.
- [11] M. L. V. Pitteway, P. M. Olive: Computer Graphics Forum, Vol. 4 (1985) No. 2, p.111.
- [12] S. Gupta, R. Sproull: Computer Graphics, Vol. 15 (1981) No.3, p.1.
- [13] X. Wu, J. G. Rokne: Computer Vision: Graphics and Image Processing, Vol. 37 (1987), No.(3), p.331.
- [14] Y. K. Liu, J. Y. Shi: Journal of Computer-aided Design & Computer Graphics, Vol.17 (2005) No.1, p.34.
- [15] X. Jiang, H. C. Zhang and Y. Z. Jing: Journal of Nanjing University of Aeronautics & Astronautics, Vol. 35 (2003) No.2, p.148.

Mechatronics and Intelligent Materials II

10.4028/www.scientific.net/AMR.490-495

Integral Algorithm for Generating Anti-Aliasing Circle Based on Bresenham Algorithm

10.4028/www.scientific.net/AMR.490-495.1202